Resumen: Acceso a datos con JPA

Unidad 3 · Spring Data JPA · Lo visto en clase y en los ejercicios de mapeo (actualizado a 07/11/2026)

1. Crear y configurar el proyecto

1.1 Spring Starter Project

Campo	Valor	Por qué
Type	**Maven**	Con Gradle exige tener instalado exactamente el Java elegido (error *ToolchainProvisioningException*).
Java Version	**21**	La que está instalada en el ordenador.
Group / Package	**com.daw**	Igual que el resto de proyectos de clase.
Spring Boot	La misma que un proyecto que ya funcione	Así puedes copiar dependencias (springdoc) sin problemas de versiones.

Dependencias que hay que marcar:

Dependencia	Para qué
Spring Web	Controladores REST (@RestController, @GetMapping...)
Spring Data JPA	Entidades (@Entity) y repositorios (JpaRepository)
MySQL Driver	Conectar con el MySQL/MariaDB de XAMPP
Validation	@NotNull, @Size, @Valid...
springdoc (Swagger)	**No sale en el asistente**: se copia a mano en el pom.xml. Boot 3.x usa springdoc 2.x; Boot 4.x usa springdoc 3.x

1.2 application.properties

```
spring.application.name=Ejercicio1Mapeo
server.port=9999

spring.datasource.url=jdbc:mysql://localhost:3306/ejercicio1_mapeo
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

- En XAMPP el usuario por defecto es **root sin contraseña**: la línea `password=` se deja **vacía**, no se borra.
- La base de datos (`ejercicio1_mapeo`) hay que **crearla antes** en phpMyAdmin.
- **No pongas** `spring.jpa.database-platform` con dialectos antiguos (MySQL5InnoDBDialect, MySQL57Dialect...): ya no existen. Si hace falta, usa `org.hibernate.dialect.MySQLDialect`, o quita la línea y Hibernate lo detecta solo.

ddl-auto	Qué hace
update	Crea las tablas que falten y añade o modifica columnas para que coincidan con las entidades. **Cambia tu tabla**.
create	Borra y crea todas las tablas en cada arranque (se pierden los datos).

ddl-auto	Qué hace
validate	Solo comprueba que las tablas coinciden; si no, no arranca.
none	No toca la base de datos.

La clase principal manda. Spring solo busca entidades, repositorios y controladores en el **paquete de la clase @SpringBootApplication y sus subpaquetes**. Si la clase principal está en `com.example.demo` y las entidades en `com.daw.datamodel.entity`, **no se crean las tablas** y no da error. Mueve la clase principal a `com.daw`.

2. La entidad

```
@Entity
@Table(name = "items")
public class Item {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false, length = 100)
    private String nombre;

    @Column(name = "price", precision = 10, scale = 2)
    private BigDecimal precio;

    @Column(name = "category")
    @Enumerated(EnumType.STRING)
    private Category categoria;

    @Lob
    @Column(name = "imagen", columnDefinition = "LONGBLOB")
    private byte[] imagen;

    // constructor vacío (si hay otros constructores) + getters y setters
}
```

Anotación	Para qué	Detalles
@Entity	La clase es una tabla	Necesita constructor vacío y getters/setters.
@Table(name)	Nombre de la tabla	Normalmente en plural: items, courses...
@Id	Clave primaria	**Solo uno por entidad.** Usa Long (wrapper), no long.
@GeneratedValue	Cómo se genera el id	**IDENTITY** para MySQL (AUTO_INCREMENT). Otras: SEQUENCE, TABLE, AUTO (la menos recomendable). Solo una columna autoincremental por tabla.
@Column	Nombre y restricciones de la columna	name, nullable, unique, length, precision, scale, insertable, updatable, columnDefinition
@Enumerated(EnumType.STRING)	Guardar un enum como texto	Sin STRING guarda la posición (0, 1, 2...) y falla con columnas ENUM de MySQL.
@Lob	Objetos pesados (imágenes, textos largos)	En Hibernate 6 un byte[] crea un **tinyblob (255 bytes)**: añade columnDefinition = "LONGBLOB".
@Basic(fetch = ...)	Carga LAZY o EAGER de un campo	Se suele usar junto a @Lob.

2.1 Tipos recomendados

Columna MySQL	Tipo Java
bigint AUTO_INCREMENT	Long
varchar / text	String (text: columnDefinition = "TEXT")
decimal(10,2) (dinero)	BigDecimal (no Float/Double)
int	Integer
tinyint(1)	Boolean
date	LocalDate
datetime / timestamp	LocalDateTime
enum('A','B')	Un enum Java + @Enumerated(EnumType.STRING)

2.2 Nombres de columnas

- Atributos Java en **camelCase** y en el idioma que quieras; el nombre real de la columna va en `@Column(name = "...")`.
- Spring convierte camelCase a minúsculas con guion bajo: `m2Tot` pasa a `m2_tot`, `typeDoc` a `type_doc`, `birthDate` a `birth_date`. Tenlo en cuenta al escribir SQL a mano.
- **length, nullable, unique, precision y scale solo se aplican cuando JPA crea o modifica la tabla** (ddl-auto). Si la tabla ya existe, lo que importa es *name*.

3. Relaciones entre entidades

Relación	Lado que tiene la FK (propietario)	Otro lado	Tabla extra
N:1 / 1:N	**El de "muchos"**: @ManyToOne + @JoinColumn	@OneToMany(mappedBy = "...")	No
N:M	Uno cualquiera: @ManyToMany + @JoinTable	@ManyToMany(mappedBy = "...")	Sí (tabla intermedia)

> **mappedBy = nombre del ATRIBUTO Java del otro lado**, no de una columna ni de una tabla. Si no coincide exactamente, la aplicación no arranca.

3.1 N:1 — Un curso tiene un instructor; un instructor da muchos cursos

```
// Curso (lado "muchos": tiene la columna id_instructor)
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "id_instructor", referencedColumnName = "id", nullable = false,
            foreignKey = @ForeignKey(name = "fk_instructor_curso"))
private Instructor instructor;

// Instructor (lado "uno")
@OneToMany(mappedBy = "instructor", fetch = FetchType.LAZY)
@JsonIgnore
private List<Curso> cursos;
```

3.2 N:M — Un curso tiene muchos alumnos; un alumno, muchos cursos

```
// Curso (lado propietario: define la tabla intermedia)
@ManyToMany
@JoinTable(name = "rel_course_student",
    joinColumns = @JoinColumn(name = "course", nullable = false,
                              foreignKey = @ForeignKey(name = "fk_course")),
    inverseJoinColumns = @JoinColumn(name = "student", nullable = false,
                              foreignKey = @ForeignKey(name = "fk_student")))
private List<Alumno> alumnos;

// Alumno
@ManyToMany(mappedBy = "alumnos")
@JsonIgnore
private List<Curso> cursos;
```

3.3 Reglas que conviene recordar

- **Obligatoria u opcional**: si el enunciado dice que algo *debe* tener la relación ("cada persona debe vivir en un domicilio"), pon `nullable = false` en @JoinColumn y `optional = false` en @ManyToOne. No los contradigas.
- **Dos relaciones entre las mismas entidades** (Persona-Vivienda: dónde vive y de cuáles es propietaria): ponles nombres que se distingan (`vivienda`, `habitantes`, `propietarios`, `propiedades`).
- Las anotaciones van **justo encima del atributo**. Si te olvidas la línea `private Tipo nombre;`, se aplican al método siguiente.
- **Bucles en JSON**: si los dos lados de una relación se muestran, Jackson entra en un bucle infinito (*Infinite recursion / StackOverflowError*). Pon `@JsonIgnore` en uno de los dos lados (normalmente en la lista).
- **LAZY y JSON**: al devolver una entidad con una relación LAZY puede salir *No serializer found for class ...ByteBuddyInterceptor* (hibernateLazyInitializer).

4. Comprobar que el mapeo funciona

- Arranca la aplicación: con `show-sql=true` verás los `create table` en la consola.
- En phpMyAdmin deben aparecer todas las tablas, incluidas las intermedias de las N:M. En **Estructura** verás las columnas FK y en **Vista de relaciones** las claves foráneas con sus nombres.
- Inserta datos **en orden**: primero las tablas sin FK (el "uno"), luego las que dependen de ellas y por último las intermedias.
- Prueba que las FK funcionan con un INSERT que apunte a un id que no existe: debe dar **#1452 ... a foreign key constraint fails**.
- Proyecto **JPATester**: su controlador `/tester/{Item}` recibe el nombre completo de la clase entidad (`com.daw.datamodel.entity.Item`) y devuelve todas sus filas en JSON.

5. Errores que te han salido y cómo se solucionan

Error / síntoma	Causa	Solución
Unable to load class org.hibernate.dialect.MySQL5InnoDBDialect	Dialecto antiguo en application.properties (no existe en Hibernate 6/7).	Quitar la línea o usar org.hibernate.dialect.MySQLDialect.
Communications link failure / Connection refused	MySQL no está arrancado (o el puerto no coincide).	Start en MySQL desde el panel de XAMPP.
Access denied for user	Usuario/contraseña que no existen en ese MySQL.	En XAMPP: root y contraseña vacía. Ver usuarios en phpMyAdmin > Cuentas de usuarios.
Invalid default value for 'updated_at'	Columna con valor por defecto 0000-00-00 00:00:00; MySQL rechaza cualquier ALTER TABLE.	ALTER TABLE items MODIFY updated_at TIMESTAMP NULL DEFAULT NULL;
Las tablas no se crean y no hay error	La clase principal está en otro paquete (com.example.demo).	Mover la clase principal a com.daw.
ToolchainProvisioningException ... languageVersion=17	Proyecto creado con Gradle y Java 17 sin tener Java 17.	Crear el proyecto con Maven y Java 21.
Delete Resource: Problems encountered while deleting	Archivos en uso: app arrancada, carpeta abierta, OneDrive...	Parar la app y cerrar lo que use la carpeta; o borrar la carpeta a mano.
'...' is not empty al crear un proyecto	Quedan restos de un proyecto con ese nombre.	Borrar la carpeta del workspace o usar otro nombre.
#1054 La columna 'm2tot' es desconocida	Spring convirtió m2Tot en m2_tot.	Mirar en Estructura el nombre real de la columna.
Hibernate modifica columnas al arrancar (varchar(255), decimal(38,2)...)	ddl-auto=update ajusta la tabla a la entidad.	Indicar length, precision/scale o columnDefinition en @Column.
La app no arranca con dos @Id	Una entidad solo tiene un @Id y una tabla un único AUTO_INCREMENT.	Un @Id; el otro identificador con @Column(unique = true, nullable = false).
mappedBy apunta a una propiedad que no existe	El nombre del mappedBy no coincide con el atributo del otro lado.	Usar el nombre exacto del atributo Java.

6. Ejercicios resueltos

Ejercicio 1: Academia

Entidad (tabla)	Atributos	Relaciones
Curso (course)	id, nombre, cuota (BigDecimal), duracion	@ManyToOne instructor (id_instructor); @ManyToMany alumnos con @JoinTable rel_course_student
Instructor (instructor)	id, nombre, telefono	@OneToMany(mappedBy = "instructor") cursos
Alumno (student)	id, nombre, telefono	@ManyToMany(mappedBy = "alumnos") cursos

Resultado: 4 tablas (course, instructor, student, rel_course_student).

Ejercicio 2: Viviendas de la provincia

Entidad (tabla)	Atributos	Relaciones
Vivienda (house)	id, direccion, codigo (único, obligatorio), m2Tot, m2Edif	@ManyToOne municipio (obligatorio); @ManyToMany propietarios con @JoinTable rel_house_person; @OneToMany(mappedBy = "vivienda") habitantes
Persona (person)	id, tipoDoc, nroDoc, nombre, apellido, fechaNacimiento	@ManyToOne vivienda donde vive (obligatoria); @ManyToMany(mappedBy = "propietarios") viviendas que posee
Municipio (city)	id, nombre, cantHab, superficie	@OneToMany(mappedBy = "municipio") viviendas

Resultado: 4 tablas (city, house, person, rel_house_person). El apellido no está en el diagrama pero sí en el texto del enunciado: si no coinciden, inclúyelo.

Vocabulario usado en los nombres de tablas y columnas

Español	Inglés	Español	Inglés
curso	course	vivienda	dwelling / house / property
alumno	student	dirección (postal)	address
instructor	instructor	fecha de nacimiento	birth date
cuota (dinero)	fee	superficie	area
cuota (cupo)	quota	habitantes	inhabitants / population